

01 - Variables

哈尔滨工业大学 黄子扬

2026 年 6 月 6 日

Contents

1	Variable Assignment & Naming / 变量赋值与命名	2
1.1	The Assignment Operator / 赋值运算符	2
1.2	Naming Rules & Conventions / 命名规则与规范	2
2	Data Types / 数据类型百科	2
2.1	Primitive Types / 原始类型	2
2.2	The None Data Type / 特殊类型 None	3
3	Strings: Detailed Manipulation / 字符串深度解析	3
3.1	Properties Creation / 属性与创建	3
3.2	Indexing & Slicing / 索引与切片	3
3.3	Immutability / 不可变性	3
4	Operators & Type Casting / 运算符与类型转换	4
4.1	Arithmetic Operators / 算术运算	4
4.2	Type Casting / 类型强制转换	4
5	String Formatting / 字符串格式化输出	4

1 Variable Assignment & Naming / 变量赋值与命名

1.1 The Assignment Operator / 赋值运算符

【Concept / 核心概念】 How it works / 赋值逻辑

Python uses the = symbol to assign a value to a variable name. It takes the value on the **right** and assigns it to the name on the **left** [1].
Python 使用 = 符号进行赋值。它将等号右侧的值“绑定”到左侧的名字上。例如：age = 25。

1.2 Naming Rules & Conventions / 命名规则与规范

- **Valid Characters / 合法字符**: Letters (A–Z, a–z), digits (0–9), and underscores (_) [2].
- **Case Sensitivity / 大小写敏感**: age is not the same as Age [3].
- **Style / 规范**: Common practice is to use lower_case_with_underscores [2].

【Warning / 避坑指南】 Legal Restrictions / 考试必考禁忌

- **No Digit Start / 严禁数字开头**: Names cannot begin with a digit (e.g., 3text is invalid) [2, 4].
- **Reserved Words / 保留字**: Don't call variables print, for, or in, as these are built-in functions you can override [4].
- **Meaningful Names / 命名意义**: Always use names that describe the data (e.g., car_length instead of L) [4].

2 Data Types / 数据类型百科

2.1 Primitive Types / 原始类型

Python has several built-in types to represent different kinds of data [5].

Type	Python Keyword	Description	Example
Integer / 整数	int	Whole numbers / 整数	100
Float / 浮点数	float	Decimals / 小数	10.7
Boolean / 布尔值	bool	True or False (Capitalized!)	True [6]
String / 字符串	str	Text data / 引号内的文本	"Hello" [7]

2.2 The None Data Type / 特殊类型 None

【Concept / 核心概念】NoneType

The `None` keyword defines a null value or no value at all [8].

`None` 用于表示“空值”。它不等于 0，不等于 `False`，也不等于空字符串 [4]。

3 Strings: Detailed Manipulation / 字符串深度解析

3.1 Properties Creation / 属性与创建

Strings have three key properties: they contain characters, have a **length** (`len()`), and characters appear in a **sequence** with numbered positions [7, 9].

3.2 Indexing & Slicing / 索引与切片

【Concept / 核心概念】The Rule of 0 / 从 0 开始计数

In Python, counting always starts at zero (0) [10].

【Example / 实操案例】Indexing Slicing / 切片实战

```
text = "Python BootCamp"
# Indexing (Single character)
print(text)    # 'P' (The first character)
print(text[-1]) # 'p' (Negative indices count from the end)

# Slicing (Substring) [start:stop]
# Rule: Inclusive on the left, EXCLUSIVE on the right.
print(text[0:6]) # 'Python' (Returns indices 0, 1, 2, 3, 4, 5)
print(text[:6])  # Omit start: assumes 0
print(text[7:])  # Omit stop: goes to the end
```

3.3 Immutability / 不可变性

【Warning / 避坑指南】Strings Are Immutable / 字符串不可修改

You cannot change characters once a string is created [11].

试图执行 `text = 'X'` 会抛出 `TypeError`。Solution / 解决方法: Must return a copy and reassign [8].

```
name = "Emlyon"
name.upper()      # Original 'name' remains "Emlyon"!
name = name.upper() # Now 'name' is "EMLYON"
```

4 Operators & Type Casting / 运算符与类型转换

4.1 Arithmetic Operators / 算术运算

- / (Division): Standard division (e.g., $5 / 2 = 2.5$).
- // (Floor division): Integer division (e.g., $5 // 2 = 2$) [12].
- ** (Exponent): Power (e.g., $3 ** 2 = 9$).

4.2 Type Casting / 类型强制转换

Use `int()`, `str()`, `float()`, and `bool()` [13].

【Warning / 避坑指南】Truthiness / 真值判定

Almost any value is `True` if it has content. Any string is `True` except empty strings (`""`). Any number is `True` except 0 [13].

5 String Formatting / 字符串格式化输出

【Example / 实操案例】The Doctor Strange Case / 案例分析

Goal: Display *"Doctor Strange has 2 heads and 3 arms."* [14].

```
name = "Doctor Strange"; heads = 2; arms = 3

# 1. f-strings (Best practice, Python 3.6+)
print(f"{name} has {heads} heads and {arms} arms.") [15]

# 2. .format() method
print("{} has {} heads and {} arms.".format(name, heads, arms)) [15]

# 3. Concatenation (Tedious!)
print(name + " has " + str(heads) + " heads and " + str(arms) + " arms.") [14]
```